2026-11-05 좋아요의 useOptimistic

- 날짜: 2026-11-05
- 앱: `lgcns-fe-next`
- React `19.3.0`, Next.js `16.4.3`
- 기준 코드: `component/organisms/PostCard.tsx`의 `LikeButton`, `actions/postActions.ts`의 `toggleLike`, `service/postService.ts`의 `toggleLike`와 `findLikedPostIds`, `lib/postCache.ts`의 `findViewerLikedPostIds`
- 이 문서는 그 파일이 지금 디스크에 있는 동작만 적는다. 소스에 없는 소켓, 낙관적 댓글, 좋아요 애니메이션은 구현된 것처럼 쓰지 않는다.

좋아요 버튼은 서버 액션이 끝나기 전에 숫자와 눌림 상태를 먼저 그린다. 그 표시는 React의 `useOptimistic`이 폼 액션 transition 동안만 유지한다. 액션이 끝나면 화면은 서버가 다시 그린 `count`와 `liked`로 돌아온다.

1. 파일이 언제 실행되는가

`app/layout.tsx`에는 좋아요 코드를 불러오는 `<script>`가 없다. `async`, `defer`, `type="module"` 속성을 단 스크립트 태그도 없다. 아래 파일은 HTML 스크립트가 아니라 모듈 소스다.

파일	선언	실행 시점	async / defer / type="module"과의 차이
`component/organisms/PostCard.tsx`	파일 첫 줄 `"use client"`. `import { useOptimistic, useState } from "react"`. `LikeButton`은 `export`가 없는 함수 선언	서버가 카드 HTML을 만든 뒤, 브라우저가 클라이언트 번들을 받아 hydrate할 때 함수가 다시 실행된다	`"use client"`는 번들러 지시문이다. `<script type="module">`이 아니다. 이 파일 경로를 `src`로 받는 태그는 없다
`actions/postActions.ts`	파일 첫 줄 `"use server"`. `export async function toggleLike`	브라우저가 폼을 제출할 때 Next가 이 함수를 서버에서 호출한다. 페이지를 열 때 버튼 핸들러로 실행되지 않는다	스크립트 태그의 `async`와 이름이 겹칠 뿐, HTML의 `async` 속성이 아니다. 함수의 `async`는 ECMAScript의 비동기 함수다
`service/postService.ts`	`async function toggleLike`, `async function findLikedPostIds`. 객체 `postService`의 메서드로만 내보낸다	서버에서 `postActions` 또는 `postCache`가 호출할 때 Prisma 쿼리가 실행된다	브라우저 모듈 그래프에 들어가지 않는다

파일	선언	실행 시점	async / defer / type="module"과의 차이
lib/postCache.ts	export async function findViewerLikedPostIds. 이 함수에는 "use cache"가 없다. 같은 파일의 getCachedPosts와 getCachedPost에는 있다	목록·상세 서버 컴포넌트가 요청마다 호출한다. 결과 메모리는 사용자별로 캐시하지 않는다	"use cache"가 있는 함수와 달리, 이 함수 본문은 캐시 히트로 건너뛰지 않는다
component/organisms/PostListSection.tsx	export default async function. "use client" 없음	서버에서 readCurrentUser와 글 목록을 기다린 뒤, 로그인 사용자면 findViewerLikedPostIds를 호출하고 PostCard에 liked를 넘긴다	서버 컴포넌트다. 브라우저가 이 파일을 defer로 실행하지 않는다
app/(post)/postdetail/[postId]/page.tsx	async function PostDetail. 페이지 기본 export는 async가 아니다	Suspense 안의 PostDetail이 params를 기다린 뒤 같은 liked 조회를 한다	페이지 파일 자체는 스크립트 src가 아니다

HTML에서 async 스크립트는 받는 즉시 실행하고 문서 파싱을 막지 않으며 순서가 없다. defer는 파싱을 막지 않고, 문서 파싱이 끝난 뒤 적힌 순서대로 실행한다. type="module"은 지연 실행에 import/export가 더해지고, 기본적으로 defer처럼 파싱 뒤에 실행된다. 이 앱의 좋아요 경로는 그 세 속성으로 붙지 않는다. 서버 컴포넌트는 요청을 그릴 때 서버에서 실행되고, PostCard의 클릭 이후 로직은 클라이언트 번들이 hydrate된 뒤에 실행된다.

참고: HTML script 요소, MDN script, ECMA-262 비동기 함수, Next.js 서버 컴포넌트와 클라이언트 컴포넌트.

2. 폼이 하나인 이유

LikeButton은 <form>을 반환한다. PostCard는 그 컴포넌트를 좋아요 자리의 유일한 폼으로 둔다. 댓글 폼과 삭제 폼은 그 폼의 자식이 아니라 형제다.

HTML 명세에서 form의 내용 모델은 플로우 콘텐츠이되, 자손으로 다른 form을 두지 못한다. 브라우저가 중첩된 폼을 만나면 안쪽 폼을 밖으로 빼거나 트리를 고쳐, 서버가 보낸 HTML과 클라이언트가 만든 트리가 달라진다. React는 그 차이를 hydration 오류로 보고한다.

이전에는 action={toggleLike}인 폼이 LikeButton의 폼을 감싸고 있었다. 그 중첩이 <form> cannot be a descendant of <form> 오류였다. 지금 바깥 폼은 없다. postId는 LikeButton 안 hidden input 하나다.

참고: HTML form 요소, MDN form, Next.js hydration 오류.

3. useOptimistic이 숫자를 먼저 바꾸는 방식

`LikeButton`의 기준값은 서버가 넘긴 `count`와 `liked`다.

```
const [like, setLike] = useOptimistic({ count, liked }, applyLike);
```

`applyLike`은 reducer다. 다음 눌림 값이 현재와 같으면 상태를 그대로 둔다. 다르면 `liked`를 바꾸고, 누르면 `count`에 1을 더하고 취소하면 1을 뺀다. 0 아래로 내려가지 않는다.

```
action={async (formData) => {
  setLike(!like.liked);
  await toggleLike(formData);
}}
```

React 19는 함수 `action`을 transition 안에서 호출한다. `setLike`이 그 안에서 불리므로, transition이 끝나기 전까지 버튼은 `like.count`와 `like.liked`를 그린다. `await toggleLike`이 끝나면 transition이 끝나고, 화면은 다시 props인 `count`와 `liked`를 따른다. props가 새 서버 렌더로 바뀌어 있으면 그 값이 남고, 액션이 실패해 props가 그대로면 누르기 전 숫자로 돌아온다.

`toggleLike` 서버 액션은 `result.ok`가 아니면 `updateTag` 없이 `return`한다. 로그인되지 않았으면 `author`가 로그인 페이지로 `redirect`한다. 둘 다 소스에 있는 종료다. 별도의 오류 토스트는 없다.

제출 버튼은 `LikeButton`이 아니라 자식 `LikeSubmit`에 있다. `useFormStatus`는 그 자식이 자기 부모 폼의 `pending`을 읽게 한다. `pending`이면 버튼을 `disabled`로 두어, 같은 액션이 끝나기 전에 한 번 더 제출되지 않는다. `aria-pressed`는 낙관적 `liked`다. 눌린 동안 배경은 `#2563eb`이다.

참고: React useOptimistic, React form, React useFormStatus.

4. 서버가 숫자를 확정하는 경로

1. `PostListSection` 또는 `PostDetail`이 `findViewerLikedPostIds`를 호출한다.
2. `lib/postCache.ts`의 그 함수는 캐시 지시문 없이 `postService.findLikedPostIds`를 호출한다.
3. 서비스는 `userId`와 화면의 `postId` 목록으로 `like.findMany`를 하고 `postId` 문자열 배열을 반환한다. 로그인 사용자가 없으면 이 조회를 호출하지 않고 `liked`는 `false`다.
4. 사용자가 버튼을 누르면 클라이언트 액션이 `setLike` 다음 `toggleLike(formData)`를 호출한다.
5. 서버 액션은 `FormData`에서 `postId`를 읽고, 세션 사용자의 `id`로 `postService.toggleLike`을 호출한다.
6. 서비스는 `postId_userId`로 기존 좋아요를 찾는다. 있으면 `delete`, 없으면 `create`다.
7. 성공하면 `updateTag("posts")`가 글 목록·상세의 `"use cache"` 항목을 지운다. `findViewerLikedPostIds`는 그 태그의 캐시 함수가 아니므로, 다음 서버 렌더가 방금 쓴 행을 다시 읽는다.

`getCachedPosts`의 캐시 값에는 `likeCount`만 있고 사용자별 `liked`는 없다. 눌림 여부는 요청마다 위 조회 단계로 계산한다.

참고: Next.js 서버 액션, Next.js updateTag, MDN FormData.

5. 이 코드가 하지 않는 일

- 다른 사용자의 화면을 누르는 즉시 바꾸지 않는다. 그 브라우저의 갱신은 목록에 있는 `PostLiveRefresh`의 `router.refresh()`다. `useOptimistic`은 버튼을 누른 문서에만 적용된다.
- 댓글 등록과 글 삭제는 `useOptimistic`을 쓰지 않는다. 각각 자기 폼의 `action={createComment}`, `action={deletePost}`다.
- 좋아요 수를 `localStorage`에 저장하지 않는다.